Lecture 2: K-nearest neighbours

2024

Xin Li
School of Computer Science,
Beijing Institute of Technology

Inductive Learning (recap)

- Induction
  - Given a training set of examples of the form $(x, f(x))$
    - x is the input, f(x) is the output
  - Return a function *h* that approximates f
    - *h* is called the hypothesis

Supervised Learning

- Two types of problems
  1. **Classification**
  2. **Regression**

- NB: The nature (categorical or continuous) of the domain (input space) off does not matter

Classification Example

- Problem: Will you enjoy an outdoor sport based on the weather?
- Training set:

Sky	Humidity	Wind	Water	Forecast	EnjoySport
Sunny	Normal	Strong	Warm	Same	yes
Sunny	High	Strong	Warm	Same	yes
Sunny	High	Strong	Warm	Change	no
Sunny	High	Strong	Cool	Change	yes

x

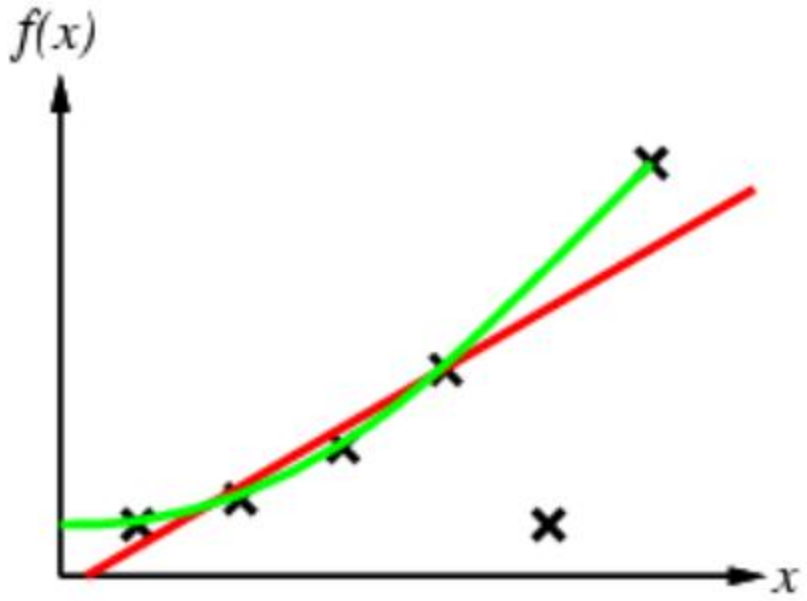
!

f(x)

- Possible Hypotheses:
  - h_1 : s = sunny → enjoysport = yes
  - h": wa = cool or F = same → enjoysport = yes

Regression Example

- Find function h that fits f at instances x



More Examples

Problem	Domain	Range	Classification / Regression
Spam Detection			
Stock price prediction			
Speech recognition			
Digit recognition			
Housing valuation			
Weather prediction			

Hypothesis Space

- Hypothesis space H
  - Set of all hypotheses *h* that the learner may consider
  - Learning is a search through hypothesis space
- Objective: find h that minimizes
  - Misclassification (or more generally some error function)

with respect to the training examples

- But what about unseen examples?

Generalization

- A good hypothesis will generalize well
  - i.e., predict unseen examples correctly

- Usually …
  - Any hypothesis *h* found to approximate the target function f well over a **sufficiently large set of training examples** will also approximate the target function well over any unobserved examples

Inductive Learning

- Goal: find an h that agrees with f on training set
  - h is **consistent** if it agrees with f on all examples

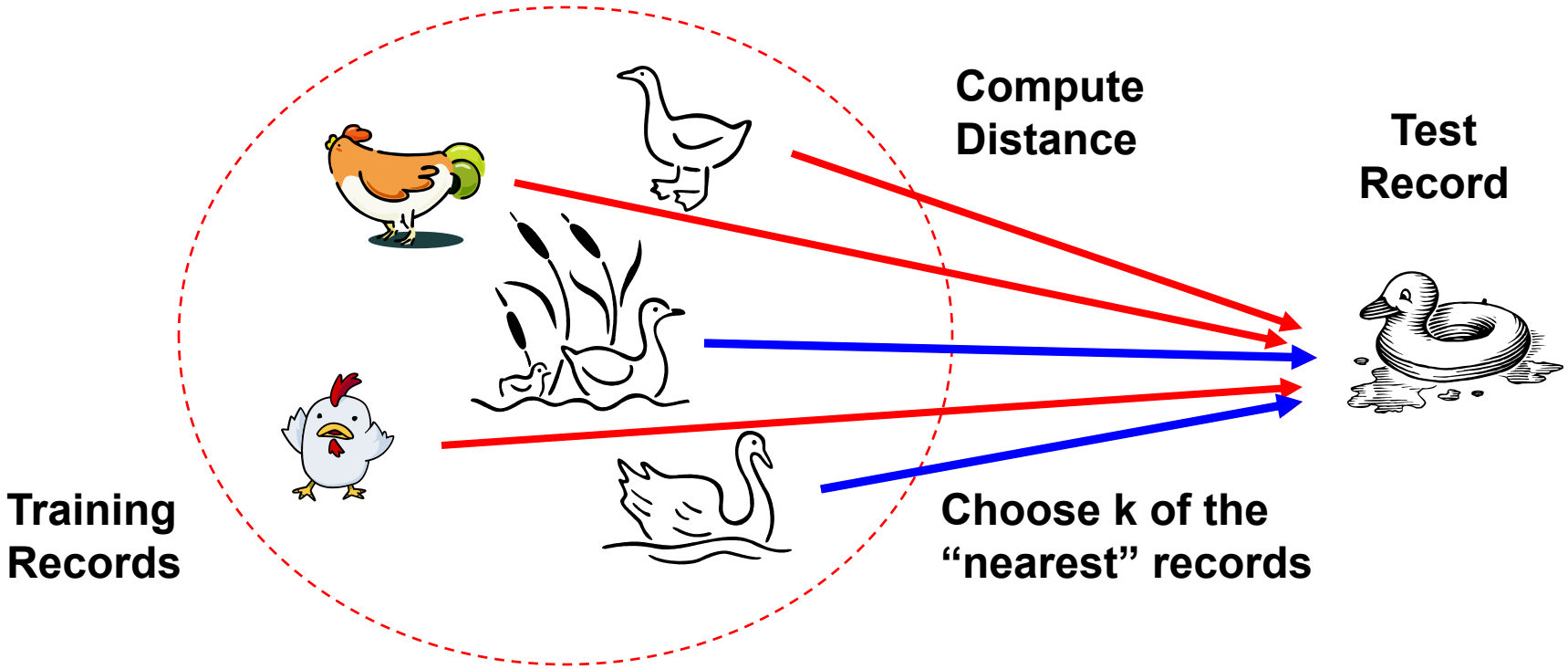
- Finding a consistent hypothesis is not always possible
  - Insufficient hypothesis space:
    - E.g., it is not possible to learn exactly $f(x) = ax + b + x\sin(x)$ when H = space of polynomials of finite degree
  - Noisy data
    - E.g., in weather prediction, identical conditions may lead to rainy and sunny days

Inductive Learning

- A learning problem is **realizable** if the hypothesis space contains the true function otherwise it is **unrealizable**.
  - Difficult to determine whether a learning problem is realizable since the true function is not known
- It is possible to use a very large hypothesis space
  - For example: H = class of all Turing machines
- But there is a **tradeoff**between **expressiveness** of a hypothesis class and the **complexity** of finding a good hypothesis

Nearest Neighbor Classifiers

Basic idea: If it walks like a duck, quacks like a duck, then it's probably a duck



Nearest Neighbour Classification

- Classification function: $h(x) = y_{x^*}$

  where y_{x^*} is the label associated with the nearest neighbour

$$x^* = argmin_{x'}\, d(x, x')$$

- Distance measures: $d(x, x')$

$$L_1: d(x, x') = \sum_j^M |x_j - x'_j|$$

$$L_2: d(x, x') = \left(\sum_j^M |x_j - x'_j|^2\right)^{1/2}$$

...

$$L_p: d(x, x') = \left(\sum_j^M |x_j - x'_j|^p\right)^{1/p}$$

Weighted dimensions: $d(x, x') = \left(\sum_j^M c_j |x_j - x'_j|^p\right)^{1/p}$

Euclidean Distance

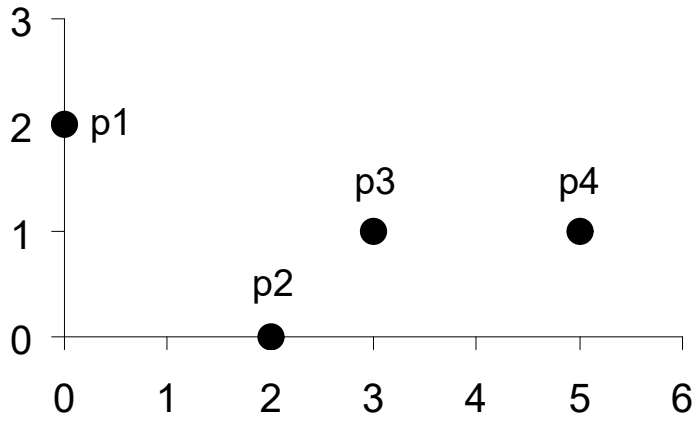
- Euclidean Distance

$$dist = \sqrt{\sum_{k=1}^{n} (p_k - q_k)^2}$$

Where n is the number of dimensions (attributes) and p_k and q_k are, respectively, the kth attributes (components) or data objects p and q.

- Standardization is necessary, if scales differ.

Euclidean Distance



point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

Distance Matrix

Minkowski Distance

- Minkowski Distance is a generalization of Euclidean Distance

$$dist = \left(\sum_{k=1}^{n} | p_k - q_k |^r \right)^{\frac{1}{r}}$$

Where r is a parameter, n is the number of dimensions (attributes) and p_k and q_k are, respectively, the kth attributes (components) or data objects p and q.

Minkowski Distance: Examples

- $r = 1$. City block (Manhattan, taxicab, L_1 norm) distance.
  - A common example of this is the Hamming distance, which is just the number of bits that are different between two binary vectors
- $r = 2$. Euclidean distance
- $r \rightarrow \infty$. "supremum" (L_{max} norm, L_∞ norm) distance.
  - This is the maximum difference between any component of the vectors
- Do not confuse *r* with *n*, i.e., all these distances are defined for all numbers of dimensions.

Minkowski Distance

point	x	y
p1	0	2
p2	2	0
p3	3	1
p4	5	1

L1	p1	p2	p3	p4
p1	0	4	4	6
p2	4	0	2	4
p3	4	2	0	2
p4	6	4	2	0

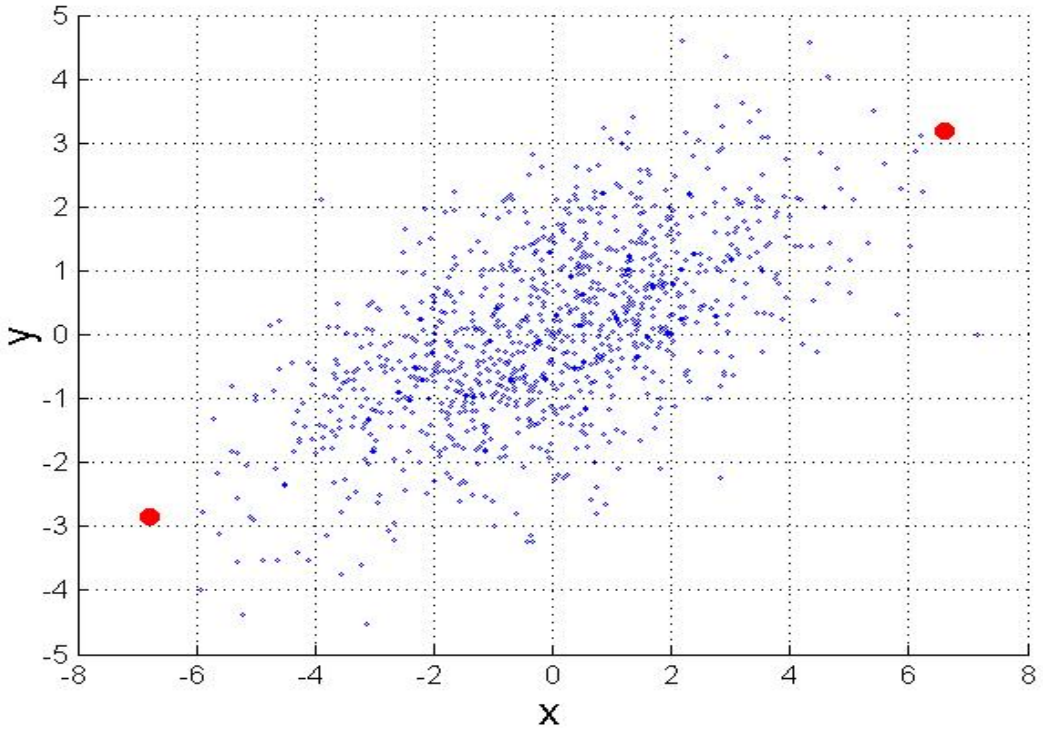
L2	p1	p2	p3	p4
p1	0	2.828	3.162	5.099
p2	2.828	0	1.414	3.162
p3	3.162	1.414	0	2
p4	5.099	3.162	2	0

L_∞	p1	p2	p3	p4
p1	0	2	3	5
p2	2	0	1	3
p3	3	1	0	2
p4	5	3	2	0

Distance Matrix

Mahalanobis Distance

$$mahalanobis(p,q) = (p-q)\Sigma^{-1}(p-q)^T$$

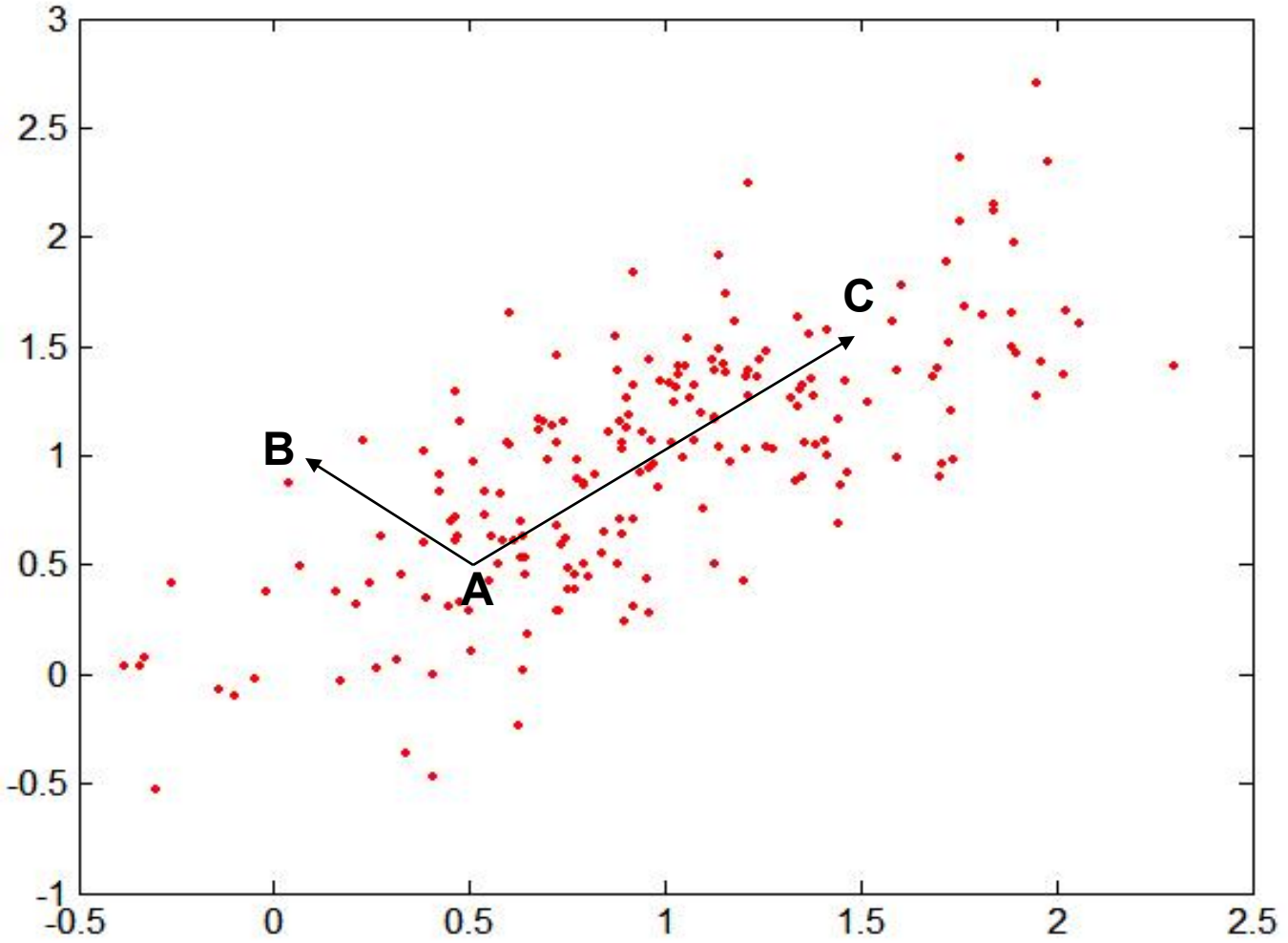


Σ is the covariance matrix of the input data *X*

$$\Sigma_{j,k} = \frac{1}{n-1}\sum_{i=1}^{n}(X_{ij} - \overline{X}_j)(X_{ik} - \overline{X}_k)$$

For red points, the Euclidean distance is 14.7, Mahalanobis distance is 6.

Mahalanobis Distance



Covariance Matrix:

$$\Sigma = \begin{bmatrix} 0.3 & 0.2 \\ 0.2 & 0.3 \end{bmatrix}$$

A: (0.5, 0.5)

B: (0, 1)

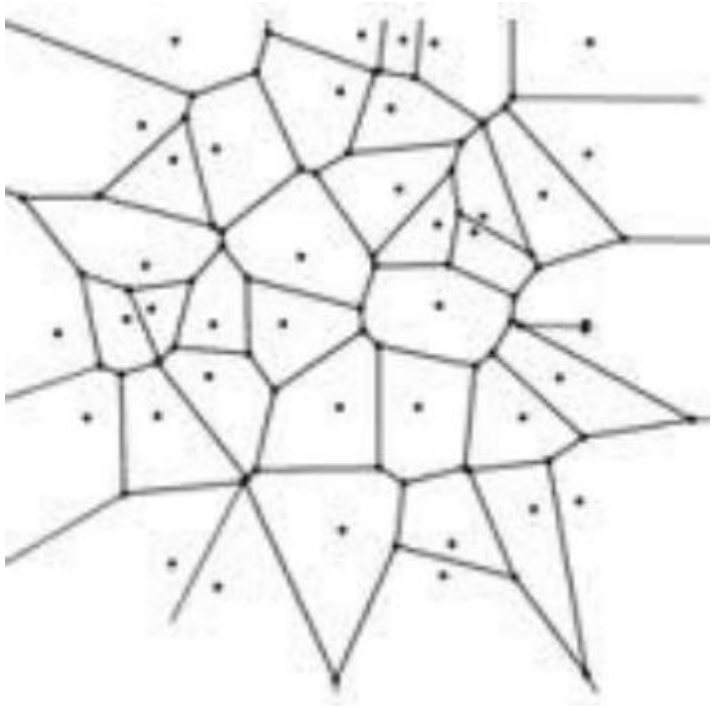
C: (1.5, 1.5)

Mahal(A,B) = 5

Mahal(A,C) = 4

Voronoi Diagram

- Partition implied by nearest neighbor fn h
  - Assuming Euclidean distance

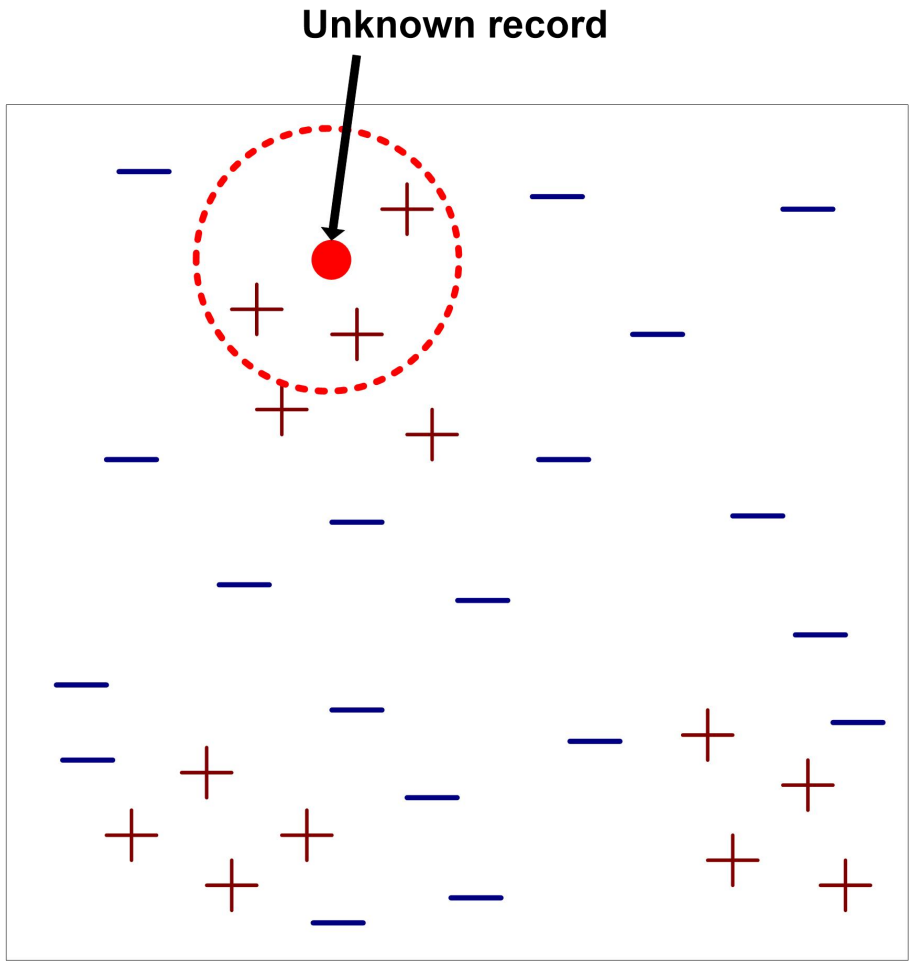


K-Nearest Neighbour

- Nearest neighbour often instable (noise)

- Idea: assign most frequent label among k-nearest neighbours
  - Let knn(x) be the k-nearest neighbours of x according to distance d
  - Label: $y_x \leftarrow mode(\{y_{x'} | x' \in knn(x)\})$

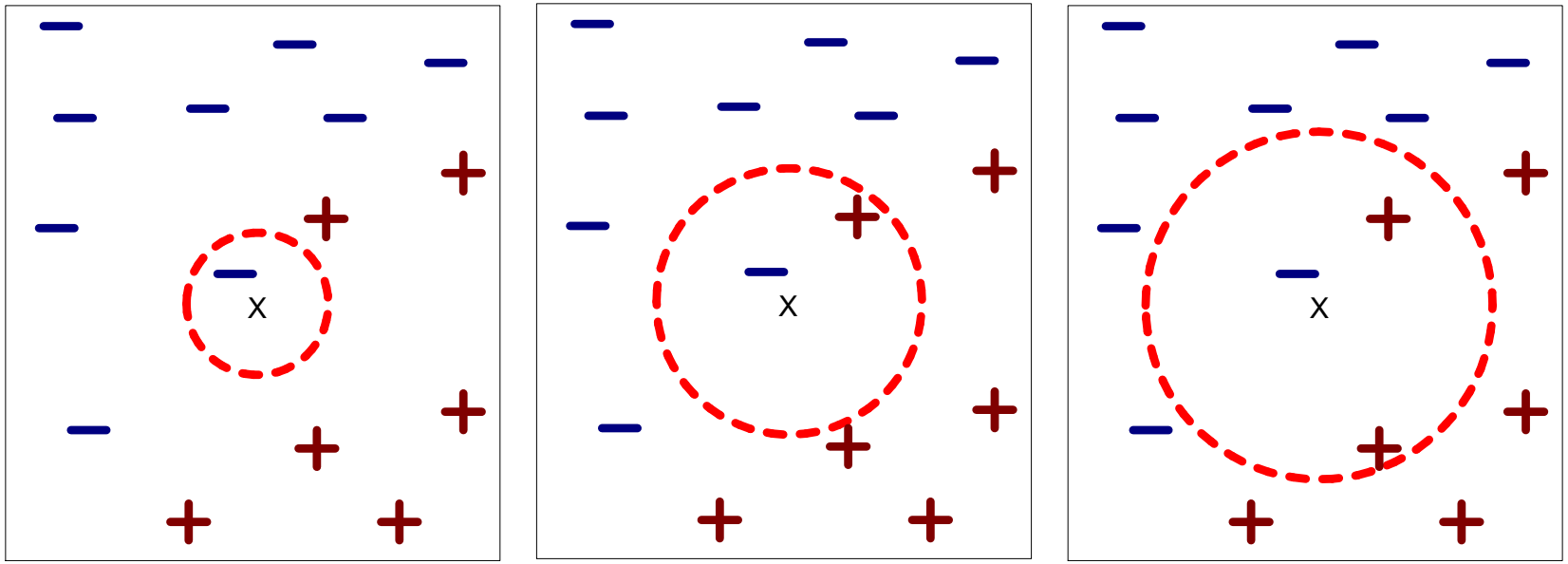
Nearest-Neighbor Classifiers



- Requires three things
  - The set of stored records
  - Distance Metric to compute distance between records
  - The value of *k*, the number of nearest neighbors to retrieve

- To classify an unknown record:
  - Compute distance to other training records
  - Identify *k* nearest neighbors
  - Use class labels of nearest neighbors to determine the class label of unknown record (e.g., by taking majority vote)

Definition of Nearest Neighbor



(a) 1-nearest neighbor (b) 2-nearest neighbor (c) 3-nearest neighbor

K-nearest neighbors of a record x are data points that have the k smallest distance to x

Nearest Neighbor Classification

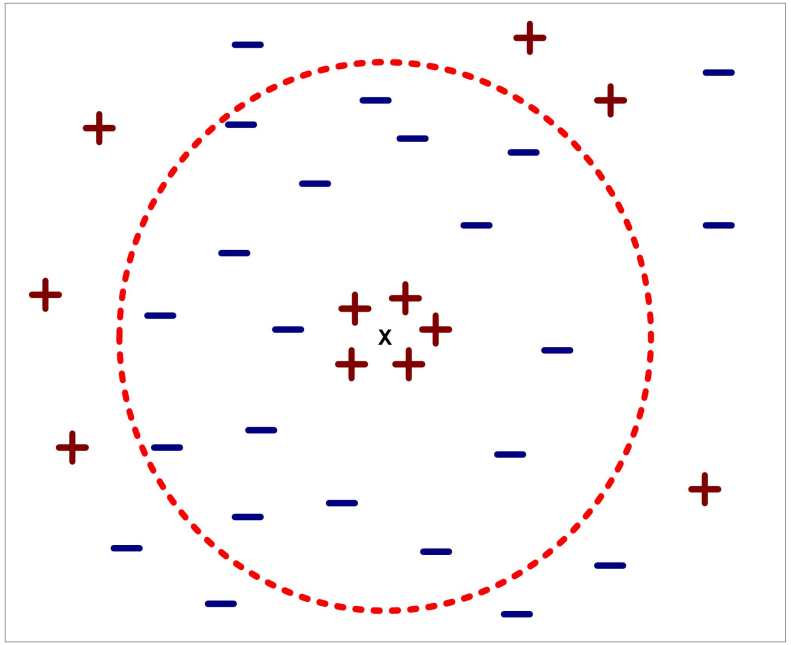
- Compute distance between two points:
  - Euclidean distance

$$d(p,q) = \sqrt{\sum_i (p_i - q_i)^2}$$

- Determine the class from nearest neighbor list
  - take the majority vote of class labels among the k-nearest neighbors
  - Weigh the vote according to distance
    - weight factor, $w = 1/d^2$

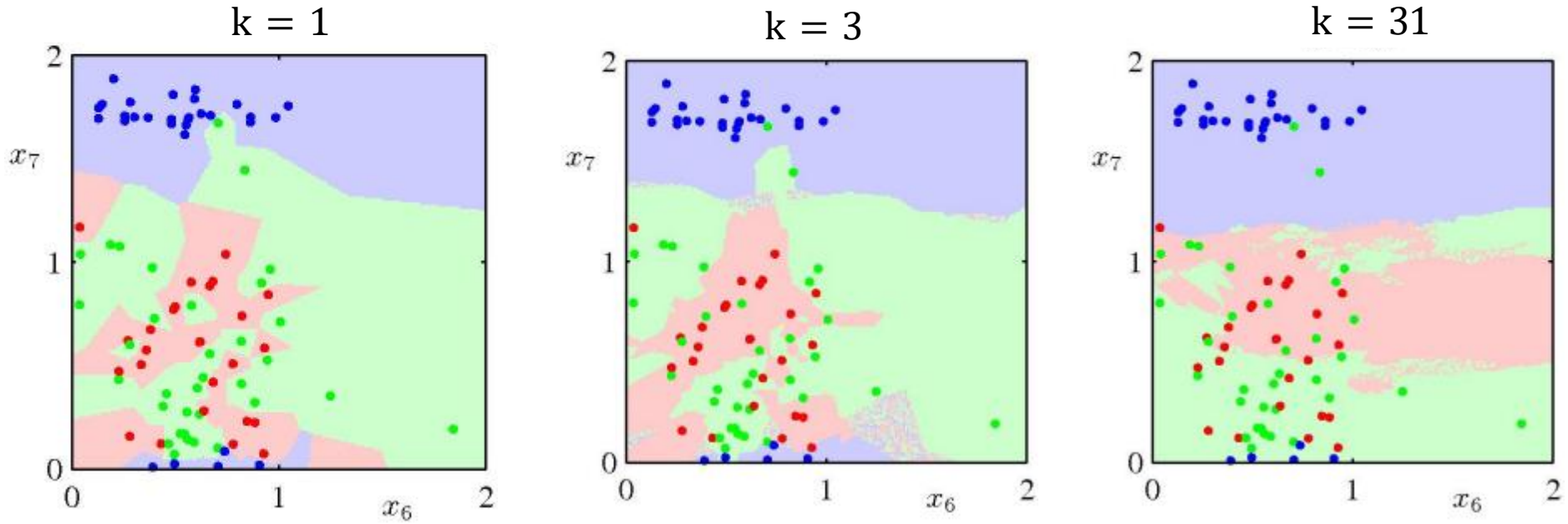
Nearest Neighbor Classification…

- Choosing the value of k:
  - If k is too small, sensitive to noise points
  - If k is too large, neighborhood may include points from other classes



Effect of K

- K controls the degree of smoothing.
- Which partition do you prefer? Why?

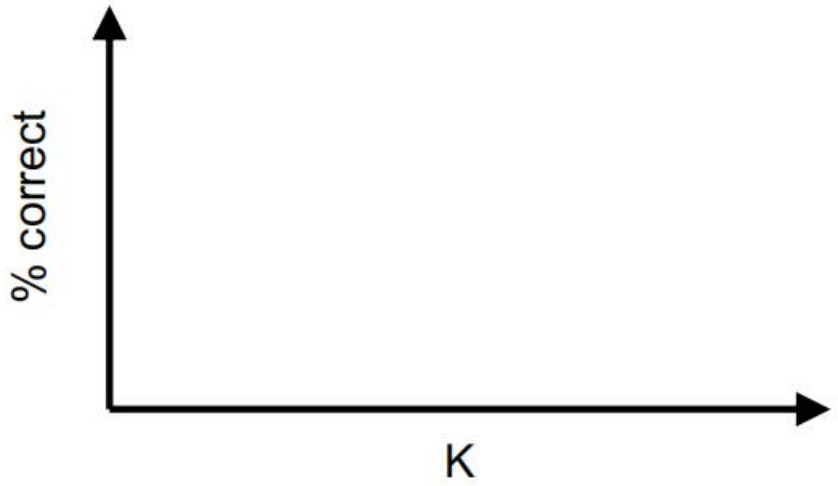


Performance of a learning algorithm

- A learning algorithm is good if it produces a hypothesis that does a good job of predicting classifications of unseen examples
- Verify performance with a **test set**
  1. Collect a large set of examples
  2. Divide into 2 disjoint sets: training set and test set
  3. Learn hypothesis *h* with training set
  4. Measure percentage of correctly classified examples by *h* in the test set

The effect of K

- Best r depends on
  - Problem
  - Amount of training data



Underfitting

- **Definition:** underfitting occurs when an algorithm finds a hypothesis h with training accuracy that is lower than the future accuracy of some other hypothesis h'

- Amount of underfitting of h:

$$max\ \{0, \max_{h'} futureAccuracy(h') - trainAccuracy(h)\}$$
$$\approx max\ \{0, \max_{h'} testAccuracy(h') - trainAccuracy(h)\}$$

- Common cause:
  - Classifier is not expressive enough

Overfitting

- **Definition:** overfitting occurs when an algorithm finds a hypothesis h with higher training accuracy than its future accuracy.

- Amount of overfitting of h:

$$\max\{0, trainAccuracy(h) - futureAccuracy(h)\}$$
$$\approx \max\{0, trainAccuracy(h) - testAccuracy(h)\}$$

- Common causes:
  - Classifier is too expressive
  - Noisy data
  - Lack of data

Choosing K

- How should we choose K?
  - Ideally: select K with highest future accuracy
  - Alternative: select K with highest test accuracy

- **Problem:** since we are choosing K based on the test set, the test set effectively becomes part of the training set when optimizing K. Hence, we cannot trust anymore the test set accuracy to be representative of future accuracy.

- Solution: split data into **training, validation and test sets**
  - **Training set:** compute nearest neighbour
  - **Validation set:** optimize hyperparameters such as K
  - **Test set:** measure performance

Choosing K based on Validation Set

Let k be the number of neighbours
For $k = 1$ to max # of neighbours
    $accuracy_k \leftarrow eval(k, trainingData, validationData)$
$k^* \leftarrow argmax_k\ accuracy_k$
$accuracy \leftarrow eval(k^*, trainingData \cup validationData, testData)$
Return $k^*, accuracy$

$eval(k, trainingData, dataset)$
    $error \leftarrow 0$
    For each $(x, y) \in dataset$
        Find $\{x_1, x_2, \ldots, x_k | (x_i, y_i) \in trainingData\}$ closest to x
        $y^* \leftarrow mode(\{y_1, y_2, \ldots, y_k\})$
        if $y \neq y^*$ then $error_k \leftarrow error_k + 1$
    $accuracy \leftarrow \frac{|dataset| - error}{|dataset|}$
    return $accuracy$

Robust validation

- How can we ensure that validation accuracy is representative of future accuracy?
  - Validation accuracy becomes more reliable as we increase the size of the validation set
  - However, this reduces the amount of data left for training

- Popular solution: **cross-validation**

Cross-Validation

- Repeatedly split training data in two parts, one for training and one for validation. Report the average validation accuracy.
- k-**fold cross validation**: split training data in k equal size subsets. Run k experiments, each time validating on one subset and training on the remaining subsets. Compute the average validation accuracy of the k experiments.
- Picture:

Selecting the Number of Neighbours by Cross-Validation

Let k be the number of neighbours

Let k' be the number of $trainingData$ splits

For $k = 1$ to max # of neighbours

For $i = 1$ to k' do (where i indexes $trainingData$ splits)

$accuracy_{ki} \leftarrow eval(k, trainingData_{1..i-1,i+1..k'}, validationData_i)$

$accuracy_k \leftarrow average(\{accuracy_{ki}\}_{\forall i})$

$k^* \leftarrow argmax_k\ accuracy_k$

$accuracy \leftarrow eval(k^*, trainingData_{1..k'}, testData)$

Return $k^*, accuracy$

Selecting the Hyperparameters by Cross-Validation

Let ϕ be a hyperparameter
Let k' be the number of $trainingData$ splits
Let h be a hypothesis obtained by training
For $\phi \in \{hyperparameter\ values\}$
    For $i = 1$ to k' do (where i indexes $trainingData$ splits)
        $h_{\phi i} \leftarrow train(\phi, trainingData_{1..i-1,i+1..k'})$
        $accuracy_{\phi i} \leftarrow test(h_{\phi i}, trainingData_i)$
    $accuracy_\phi \leftarrow average(\{accuracy_{\phi i}\}_{\forall i})$
$\phi^* \leftarrow argmax_\phi\ accuracy_\phi$
$h \leftarrow train(\phi^*, trainingData_{1..k'})$
$accuracy \leftarrow test(h, testData)$
Return $\phi^*, h, accuracy$

Weighted K-Nearest Neighbour

- We can often improve K-nearest neighbours by weighting each neighbour based on some distance measure

$$w(x, x') \propto \frac{1}{distance(x, x')}$$

- Label $y_x \leftarrow argmax_y \sum_{\{x' | x' \in knn(x) \wedge y = y_{x'}\}} w(x, x')$

where $knn(x)$ is the set of K nearest neighbours of x

K-Nearest Neighbour Regression

- We can also use KNN for regression
- Let y_x be a real value instead of a categorical label
- K-nearest neighbour regression:

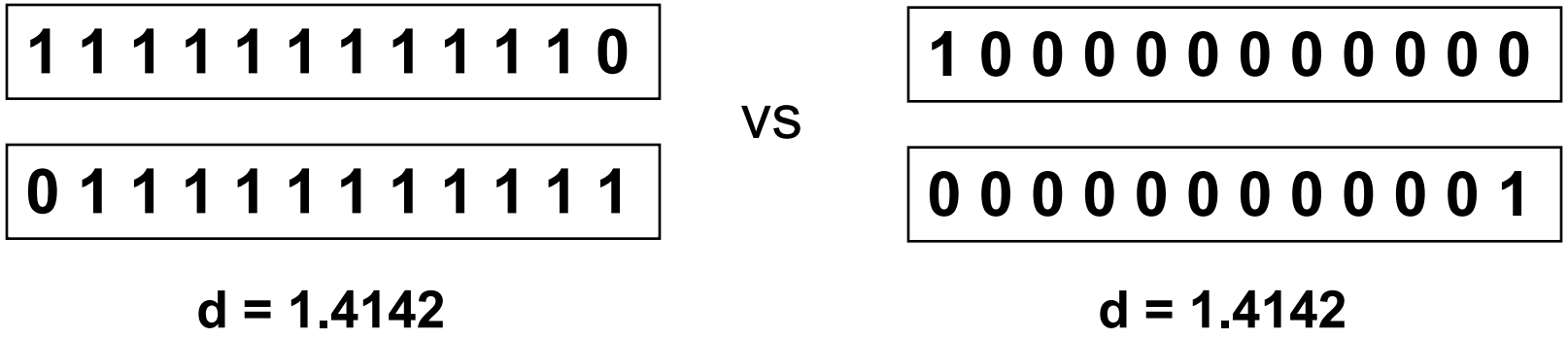
$$y_x \leftarrow average(\{y_{x'} | x' \in knn(x)\})$$

- Weighted K-nearest neighbour regression:

$$y_x \leftarrow \frac{\sum_{x' \in knn(x)} w(x, x') y_{x'}}{\sum_{x' \in knn(x)} w(x, x')}$$

Nearest neighbor Classification…

- Problem with Euclidean measure:
  - High dimensional data
    - curse of dimensionality
  - Can produce counter-intuitive results



Nearest neighbor Classification…

- k-NN classifiers are lazy learners
  - It does not build models explicitly
  - Unlike eager learners such as decision tree induction and rule-based systems
  - Classifying unknown records are relatively expensive

Bayes Classifier

- A probabilistic framework for solving classification problems
- Conditional Probability:

$$P(C \mid A) = \frac{P(A, C)}{P(A)}$$

$$P(A \mid C) = \frac{P(A, C)}{P(C)}$$

- Bayes theorem:

$$P(C \mid A) = \frac{P(A \mid C)P(C)}{P(A)}$$

Example of Bayes Theorem

- Given:
  - A doctor knows that meningitis causes stiff neck 50% of the time
  - Prior probability of any patient having meningitis is 1/50,000
  - Prior probability of any patient having stiff neck is 1/20

- If a patient has stiff neck, what's the probability he/she has meningitis?

$$P(M \mid S) = \frac{P(S \mid M)P(M)}{P(S)} = \frac{0.5 \times 1/50000}{1/20} = 0.0002$$

Bayesian Classifiers

- Consider each attribute and class label as random variables
- Given a record with attributes (A_1, A_2,…,A_n)
  - Goal is to predict class C
  - Specifically, we want to find the value of C that maximizes P(C| A_1, A_2,…,A_n)
- Can we estimate P(C| A_1, A_2,…,A_n) directly from data?

Bayesian Classifiers

- Approach:
  - compute the posterior probability $P(C \mid A_1, A_2, \ldots, A_n)$ for all values of C using the Bayes theorem

$$P(C \mid A_1 A_2 \ldots A_n) = \frac{P(A_1 A_2 \ldots A_n \mid C) P(C)}{P(A_1 A_2 \ldots A_n)}$$

  - Choose value of C that maximizes
    $P(C \mid A_1, A_2, \ldots, A_n)$

  - Equivalent to choosing value of C that maximizes
    $P(A_1, A_2, \ldots, A_n \mid C)\, P(C)$

- How to estimate $P(A_1, A_2, \ldots, A_n \mid C)$?

Naïve Bayes Classifier

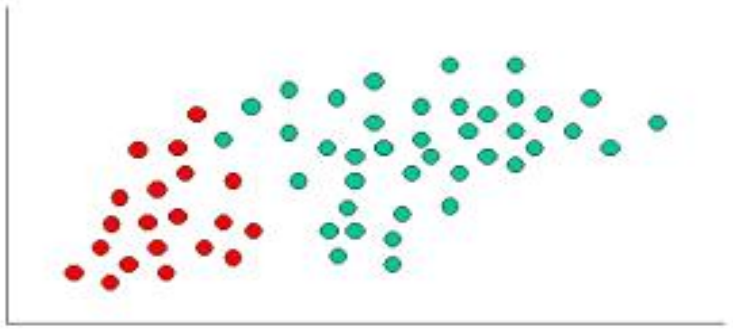
- Assume independence among attributes A_i when class is given:
  - $P(A_1, A_2, \ldots, A_n \mid C_j) = P(A_1 \mid C_j)\, P(A_2 \mid C_j) \ldots P(A_n \mid C_j)$
  - Can estimate $P(A_i \mid C_j)$ for all A_i and C_j.
  - New point is classified to C_j if $P(C_j) \prod P(A_i \mid C_j)$ is maximal.

Naive Bayes Classifier

Technical Notes

Naive Bayes Classifier Introductory Overview

The Naive Bayes Classifier technique is based on the so-called Bayesian theorem and is particularly suited when the dimensionality of the inputs is high. Despite its simplicity, Naive Bayes can often outperform more sophisticated classification methods.



To demonstrate the concept of Naïve Bayes Classification, consider the example displayed in the illustration above. As indicated, the objects can be classified as either GREEN or RED. Our task is to classify new cases as they arrive, i.e., decide to which class label they belong, based on the currently exiting objects.

Since there are twice as many GREEN objects as RED, it is reasonable to believe that a new case (which hasn't been observed yet) is twice as likely to have membership GREEN rather than RED. In the Bayesian analysis, this belief is known as the prior probability. Prior probabilities are based on previous experience, in this case the percentage of GREEN and RED objects, and often used to predict outcomes before they actually happen.

Thus, we can write:

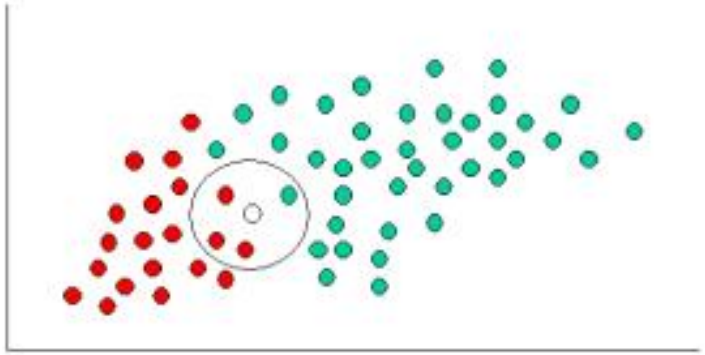
$$Prior\ probability\ for\ GREEN \propto \frac{Number\ of\ GREEN\ objects}{Total\ number\ of\ objects}$$

$$Prior\ probability\ for\ RED \propto \frac{Number\ of\ RED\ objects}{Total\ number\ of\ objects}$$

Since there is a total of 60 objects, 40 of which are GREEN and 20 RED, our prior probabilities for class membership are:

$$Prior\ probability\ for\ GREEN \propto \frac{40}{60}$$

$$Prior\ probability\ for\ RED \propto \frac{20}{60}$$



Having formulated our prior probability, we are now ready to classify a new object (WHITE circle). Since the objects are well clustered, it is reasonable to assume that the more GREEN (or RED) objects in the vicinity of X, the more likely that the new cases belong to that particular color. To measure this likelihood, we draw a circle around X which encompasses a number (to be chosen a priori) of points irrespective of their class labels. Then we calculate the number of points in the circle belonging to each class label. From this we calculate the likelihood:

$$Likelihood\ of\ X\ given\ GREEN \propto \frac{Number\ of\ GREEN\ in\ the\ vicinity\ of\ X}{Total\ number\ of\ GREEN\ cases}$$

$$Likelihood\ of\ X\ given\ RED \propto \frac{Number\ of\ RED\ in\ the\ vicinity\ of\ X}{Total\ number\ of\ RED\ cases}$$

From the illustration above, it is clear that Likelihood of X given GREEN is smaller than Likelihood of X given RED, since the circle encompasses 1 GREEN object and 3 RED ones. Thus:

$$Probability\ of\ X\ given\ GREEN \propto \frac{1}{40}$$

$$Probability\ of\ X\ given\ RED \propto \frac{3}{20}$$

Although the prior probabilities indicate that X may belong to GREEN (given that there are twice as many GREEN compared to RED) the likelihood indicates otherwise; that the class membership of X is RED (given that there are more RED objects in the vicinity of X than GREEN). In the Bayesian analysis, the final classification is produced by combining both sources of information, i.e., the prior and the likelihood, to form a posterior probability using the so-called Bayes' rule (named after Rev. Thomas Bayes 1702-1761).

$$Posterior\ probability\ of\ X\ being\ GREEN \propto$$

$$Prior\ probability\ of\ GREEN \times Likelihood\ of\ X\ given\ GREEN$$

$$= \frac{4}{6} \times \frac{1}{40} = \frac{1}{60}$$

$$Posterior\ probability\ of\ X\ being\ RED \propto$$

$$Prior\ probability\ of\ RED \times Likelihood\ of\ X\ given\ RED$$

$$= \frac{2}{6} \times \frac{3}{20} = \frac{1}{20}$$

Finally, we classify X as RED since its class membership achieves the largest posterior probability.

Note. The above probabilities are not normalized. However, this does not affect the classification outcome since their normalizing constants are the same.

Technical Notes

In the previous section, we provided an intuitive example for understanding classification using Naive Bayes. In this section are further details of the technical issues involved. Naive Bayes classifiers can handle an arbitrary number of independent variables whether continuous or categorical. Given a set of variables, X = {x1,x2,x...,xd}, we want to construct the posterior probability for the event Cj among a set of possible outcomes C = {c1,c2,c...,cd}. In a more familiar language, X is the predictors and C is the set of categorical levels present in the dependent variable. Using Bayes' rule:

$$p(C_j \mid x_1, x_2, \ldots, x_d) \propto p(x_1, x_2, \ldots, x_d \mid C_j) p(C_j)$$

where p(Cj | x1,x2,x...,xd) is the posterior probability of class membership, i.e., the probability that X belongs to Cj. Since Naive Bayes assumes that the conditional probabilities of the independent variables are statistically independent we can decompose the likelihood to a product of terms:

$$p(X \mid C_j) \propto \prod_{k=1}^{d} p(x_k \mid C_j)$$

and rewrite the posterior as:

$$p(C_j \mid X) \propto p(C_j) \prod_{k=1}^{d} p(x_k \mid C_j)$$

Using Bayes' rule above, we label a new case X with a class level Cj that achieves the highest posterior probability.

Although the assumption that the predictor (independent) variables are independent is not always accurate, it does simplify the classification task dramatically, since it allows the class conditional densities p(xk | Cj) to be calculated separately for each variable, i.e., it reduces a multidimensional task to a number of one-dimensional ones. In effect, Naive Bayes reduces a high-dimensional density estimation task to a one-dimensional kernel density estimation. Furthermore, the assumption does not seem to greatly affect the posterior probabilities, especially in regions near decision boundaries, thus, leaving the classification task unaffected.

Naive Bayes can be modeled in several different ways including normal, lognormal, gamma and Poisson density functions:

$$p(x_k \mid C_j) = \begin{cases} \dfrac{1}{\sigma_{kj}\sqrt{2\pi}} \exp\left(\dfrac{-(x-\mu_{kj})^2}{2\sigma_{kj}}\right), & -\infty < x < \infty, -\infty < \mu_{kj} <, \sigma_{kj} > 0 \quad \text{Normal} \\ \mu_{kj} : \text{mean}, \ \sigma_{kj} : \text{standard deviation} & \\ \dfrac{1}{x\sigma_{kj}(2\pi)^{1/2}} \exp\left\{\dfrac{-[\log(x/m_{kj})]^2}{2\sigma_{kj}^{\,2}}\right\}, & 0 < x < \infty, m_{kj} > 0, \sigma_{kj} > 0 \quad \text{Lognormal} \\ m_{kj} : \text{scale parameter}, \ \sigma_{kj} : \text{shape parameter} & \\ \dfrac{\left(\dfrac{x}{b_{kj}}\right)^{c_{kj}-1}}{b_{kj}\Gamma(c_{kj})} \exp\left(\dfrac{-x}{b_{kj}}\right), & 0 \le x < \infty, b_{kj} > 0, c_{kj} > 0 \quad \text{Gamma} \\ b_{kj} : \text{scale parameter}, \ c_{kj} : \text{shape parameter} & \\ \dfrac{\lambda_{kj}\exp(-\lambda_{kj})}{x!}, & 0 \le x < \infty, \lambda_{kj} > 0, x = 0,1,2,\ldots \quad \text{Poisson} \\ \lambda_{kj} : \text{mean} & \end{cases}$$

Note. Poisson variables are regarded here as continuous since they are ordinal rather than truly categorical. For categorical variables, a discrete probability is used with values of the categorical level being proportional to their conditional frequency in the training data.

How to Estimate Probabilities from Data?

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Class: $P(C) = N_c/N$
  - e.g., P(No) = 7/10,
    P(Yes) = 3/10

- For discrete attributes:

$$P(A_i \mid C_k) = |A_{ik}|/ N_{k}c$$

  - where $|A_{ik}|$ is number of instances having attribute A_i and belongs to class C_k
  - Examples:

P(Status=Married|No) = 4/7
P(Refund=Yes|Yes)=0

How to Estimate Probabilities from Data?

- For continuous attributes:
  - Discretize the range into bins
    - one ordinal attribute per bin
    - violates independence assumption
  - Two-way split: $(A < v)$ or $(A > v)$
    - choose only one of the two splits as new attribute
  - Probability density estimation:
    - Assume attribute follows a normal distribution
    - Use data to estimate parameters of distribution (e.g., mean and standard deviation)
    - Once probability distribution is known, can use it to estimate the conditional probability $P(A_i|c)$

How to Estimate Probabilities from Data?

Tid	Refund	Marital Status	Taxable Income	Evade
1	Yes	Single	125K	No
2	No	Married	100K	No
3	No	Single	70K	No
4	Yes	Married	120K	No
5	No	Divorced	95K	Yes
6	No	Married	60K	No
7	Yes	Divorced	220K	No
8	No	Single	85K	Yes
9	No	Married	75K	No
10	No	Single	90K	Yes

- Normal distribution:

$$P(A_i \mid c_j) = \frac{1}{\sqrt{2\pi\sigma_{ij}^2}} e^{-\frac{(A_i-\mu_{ij})^2}{2\sigma_{ij}^2}}$$

  - One for each (A_i,c_i) pair

- For (Income, Class=No):
  - If Class=No
    - sample mean = 110
    - sample variance = 2975

$$P(Income = 120 \mid No) = \frac{1}{\sqrt{2\pi}(54.54)} e^{-\frac{(120-110)^2}{2(2975)}} = 0.0072$$

Example of Naïve Bayes Classifier

Given a Test Record:

$$X = (\text{Refund} = \text{No}, \text{Married}, \text{Income} = 120\text{K})$$

naive Bayes Classifier:

P(Refund=Yes|No) = 3/7
P(Refund=No|No) = 4/7
P(Refund=Yes|Yes) = 0
P(Refund=No|Yes) = 1
P(Marital Status=Single|No) = 2/7
P(Marital Status=Divorced|No)=1/7
P(Marital Status=Married|No) = 4/7
P(Marital Status=Single|Yes) = 2/7
P(Marital Status=Divorced|Yes)=1/7
P(Marital Status=Married|Yes) = 0

For taxable income:
If class=No: sample mean=110
sample variance=2975
If class=Yes: sample mean=90
sample variance=25

- P(X|Class=No) = P(Refund=No|Class=No)
  × P(Married| Class=No)
  × P(Income=120K| Class=No)
  = 4/7 × 4/7 × 0.0072 = 0.0024

- P(X|Class=Yes) = P(Refund=No| Class=Yes)
  × P(Married| Class=Yes)
  × P(Income=120K| Class=Yes)
  = 1 × 0 × 1.2 × 10^{-9} = 0

Since P(X|No)P(No) > P(X|Yes)P(Yes)

Therefore P(No|X) > P(Yes|X)
=> Class = No

Example of Naïve Bayes Classifier

Name	Give Birth	Can Fly	Live in Water	Have Legs	Class
human	yes	no	no	yes	mammals
python	no	no	no	no	non-mammals
salmon	no	no	yes	no	non-mammals
whale	yes	no	yes	no	mammals
frog	no	no	sometimes	yes	non-mammals
komodo	no	no	no	yes	non-mammals
bat	yes	yes	no	yes	mammals
pigeon	no	yes	no	yes	non-mammals
cat	yes	no	no	yes	mammals
leopard shark	yes	no	yes	no	non-mammals
turtle	no	no	sometimes	yes	non-mammals
penguin	no	no	sometimes	yes	non-mammals
porcupine	yes	no	no	yes	mammals
eel	no	no	yes	no	non-mammals
salamander	no	no	sometimes	yes	non-mammals
gila monster	no	no	no	yes	non-mammals
platypus	no	no	no	yes	mammals
owl	no	yes	no	yes	non-mammals
dolphin	yes	no	yes	no	mammals
eagle	no	yes	no	yes	non-mammals

A: attributes

M: mammals

N: non-mammals

$$P(A \mid M) = \frac{6}{7} \times \frac{6}{7} \times \frac{2}{7} \times \frac{2}{7} = 0.06$$

$$P(A \mid N) = \frac{1}{13} \times \frac{10}{13} \times \frac{3}{13} \times \frac{4}{13} = 0.0042$$

$$P(A \mid M)P(M) = 0.06 \times \frac{7}{20} = 0.021$$

$$P(A \mid N)P(N) = 0.004 \times \frac{13}{20} = 0.0027$$

Give Birth	Can Fly	Live in Water	Have Legs	Class
yes	no	yes	no	?

P(A|M)P(M) > P(A|N)P(N)

=> Mammals

Naïve Bayes Classifier

- If one of the conditional probability is zero, then the entire expression becomes zero
  - Due to data samples not covering variables well
- Probability estimation:

$$\text{Original}: P(A_i \mid C) = \frac{N_{ic}}{N_c}$$

$$\text{Laplace}: P(A_i \mid C) = \frac{N_{ic} + 1}{N_c + c}$$

$$\text{m - estimate}: P(A_i \mid C) = \frac{N_{ic} + mp}{N_c + m}$$

c: number of classes

p: parameter. Equal to prior probability in case of no training data

m: parameter

Naïve Bayes (Summary)

- Robust to isolated noise points
- Handle missing values by ignoring the instance during probability estimate calculations
- Robust to irrelevant attributes
- Independence assumption may not hold for some attributes
  - Use other techniques such as Bayesian Belief Networks (BBN)